

Class Activity-4

Name: R. Reddy Balaji

Roll No: 192424122

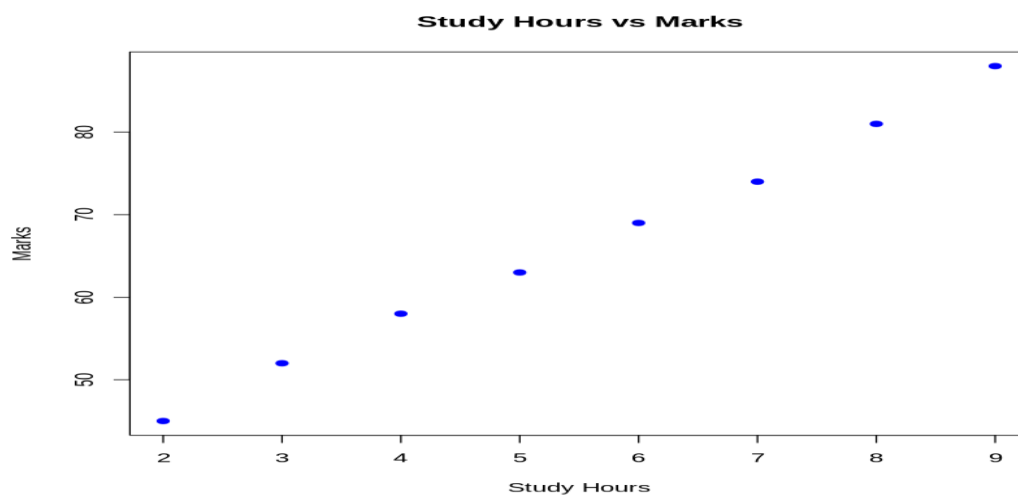
DSA0602 Data Handling and Visualization

1. Study Hours vs Marks (R)

R Program

```
# Create dataset  
StudyHours <- c(2,3,4,5,6,7,8,9)  
Marks <- c(45,52,58,63,69,74,81,88)  
data <- data.frame(StudyHours, Marks)  
  
# Scatterplot  
plot(data$StudyHours, data$Marks,  
      main="Study Hours vs Marks",  
      xlab="Study Hours",  
      ylab="Marks",  
      pch=19,  
      col="blue")
```

Output



Interpretation

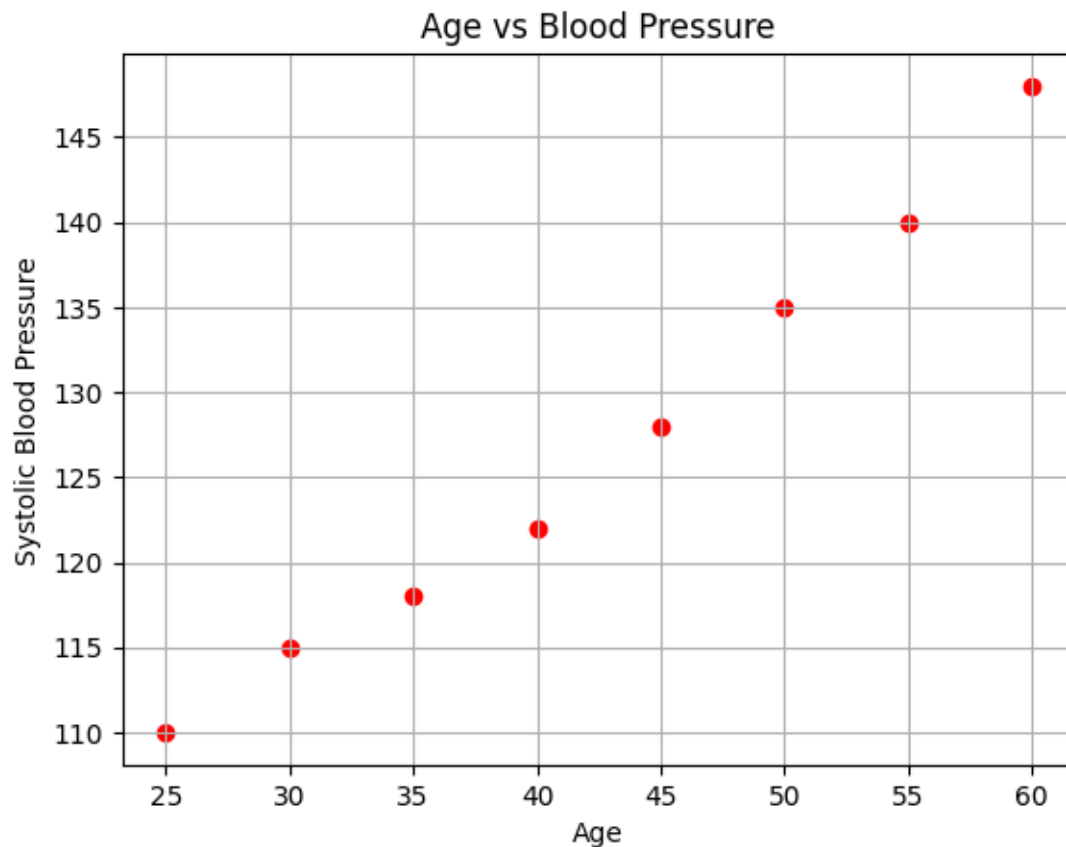
There is a strong positive correlation between study hours and marks. Students who study more tend to obtain higher marks.

2. Age vs Blood Pressure (Python)

Python Program

```
import matplotlib.pyplot as plt  
age = [25,30,35,40,45,50,55,60]  
bp = [110,115,118,122,128,135,140,148]  
plt.scatter(age, bp, color='red')  
plt.xlabel("Age")  
plt.ylabel("Systolic Blood Pressure")  
plt.title("Age vs Blood Pressure")  
plt.grid(True)  
plt.show()
```

Output



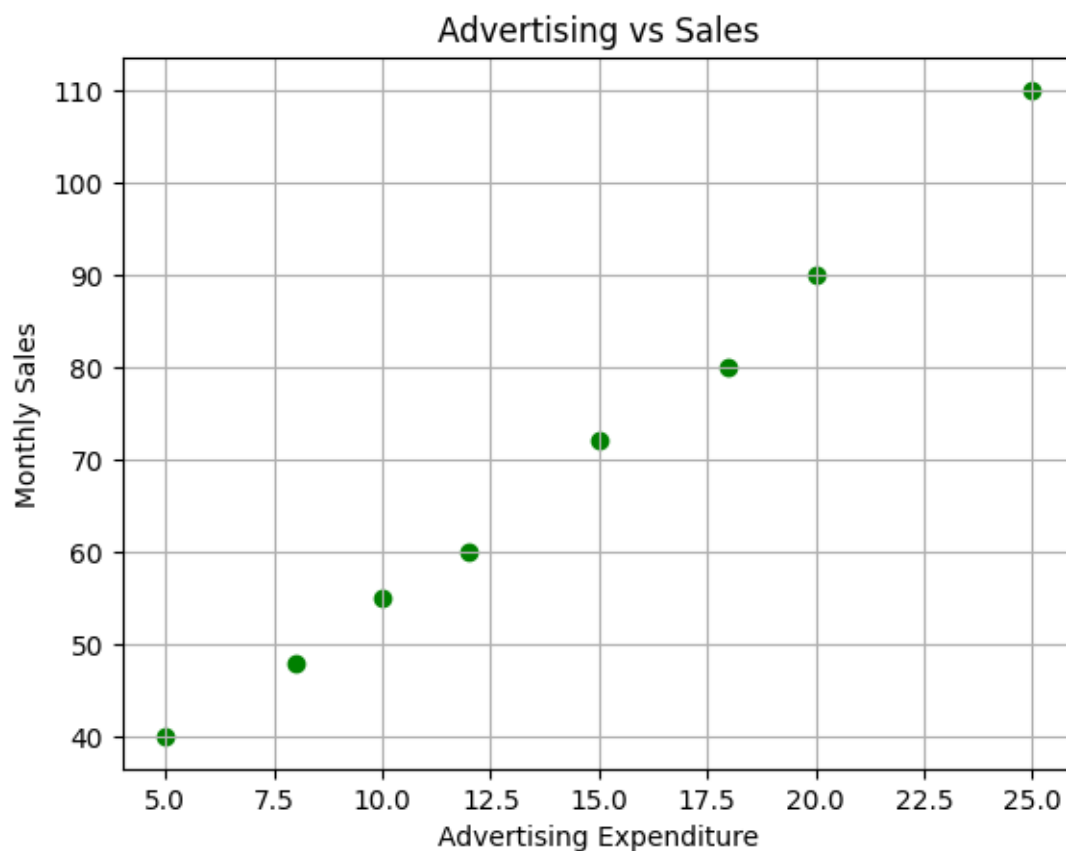
Observation

As age increases, systolic blood pressure generally increases, indicating a positive trend.

3. Advertising Expenditure vs Monthly Sales (Python)

```
import matplotlib.pyplot as plt  
advertising = [5,8,10,12,15,18,20,25]  
sales = [40,48,55,60,72,80,90,110]  
plt.scatter(advertising, sales, color='green')  
plt.xlabel("Advertising Expenditure")  
plt.ylabel("Monthly Sales")  
plt.title("Advertising vs Sales")  
plt.grid(True)  
plt.show()
```

Output



Conclusion

Monthly sales **increase** as advertising expenditure increases.

4. Correlation Matrix and Correlogram (Python)

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

data = {
    'Income':[30,40,50,60,70,80],
    'Savings':[5,10,15,20,25,30],
    'Loan Amount':[15,20,25,30,35,40],
    'Credit Score':[600,650,680,720,760,800]
}

df = pd.DataFrame(data)

corr = df.corr()

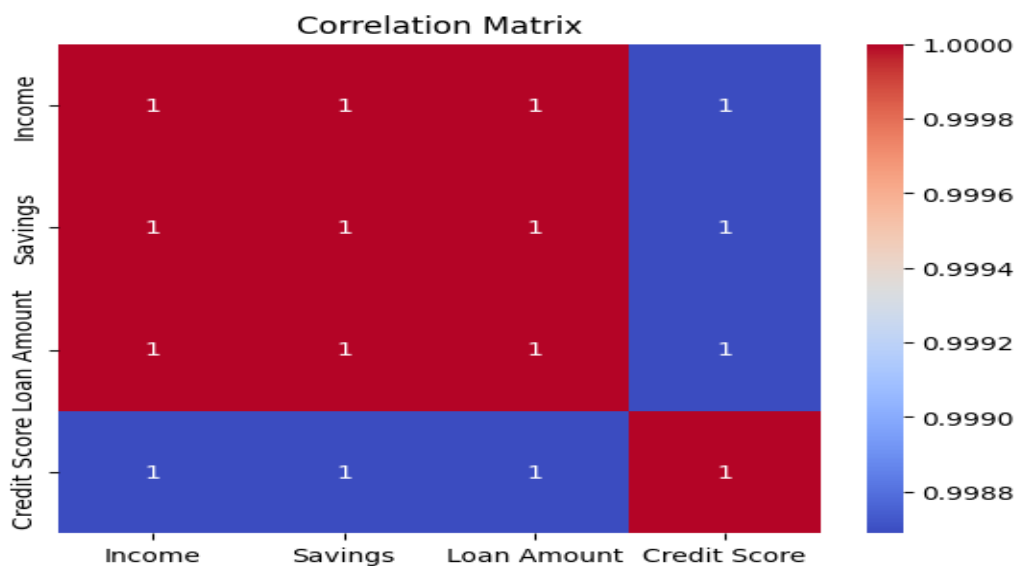
print(corr)

sns.heatmap(corr, annot=True, cmap='coolwarm')

plt.title("Correlation Matrix")

plt.show()
```

output



Answer

The Income and Savings variables exhibit the strongest positive correlation.

5. Iris Dataset Correlation Heatmap

```
from sklearn.datasets import load_iris

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

corr = df.corr()

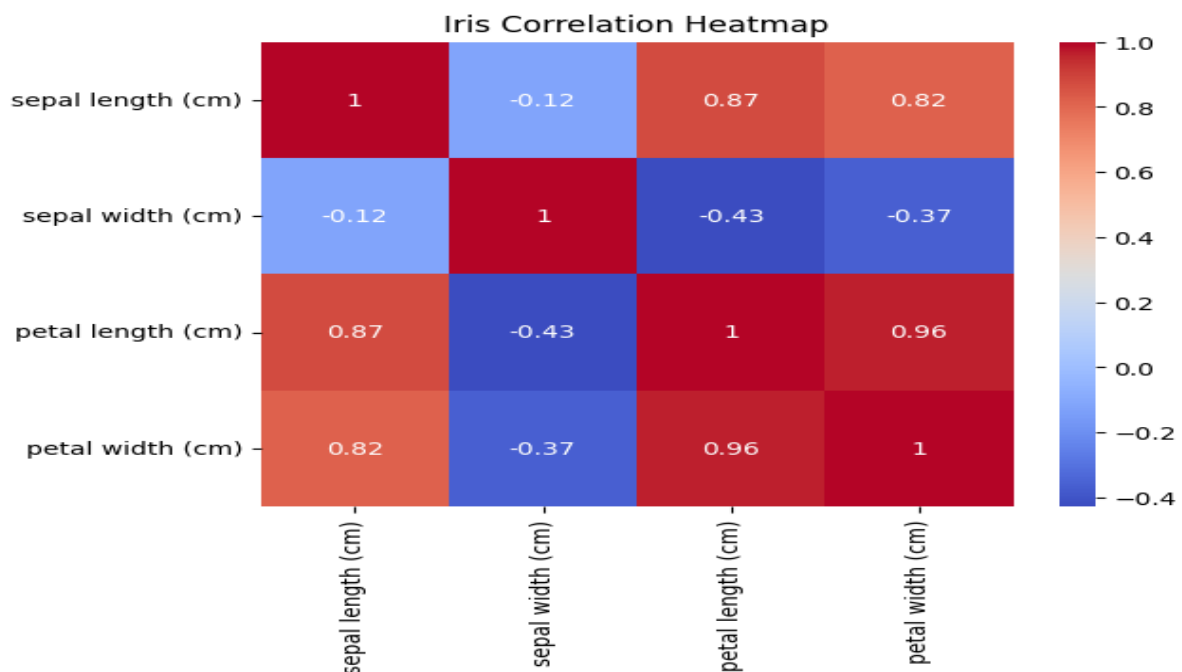
print(corr)

sns.heatmap(corr,
            annot=True,
            cmap="coolwarm")

plt.title("Iris Correlation Heatmap")

plt.show()
```

output



Answer

Negative correlation is observed mainly between

- Sepal Length and Sepal Width

6. Healthcare Correlogram

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

data = {
    'BMI':[22,24,26,28,30,32],
    'Cholesterol':[180,190,200,210,220,230],
    'Blood Pressure':[120,124,130,136,142,150],
    'Glucose':[90,95,100,110,118,126],
    'Heart Rate':[70,72,74,76,78,80]
}

df = pd.DataFrame(data)

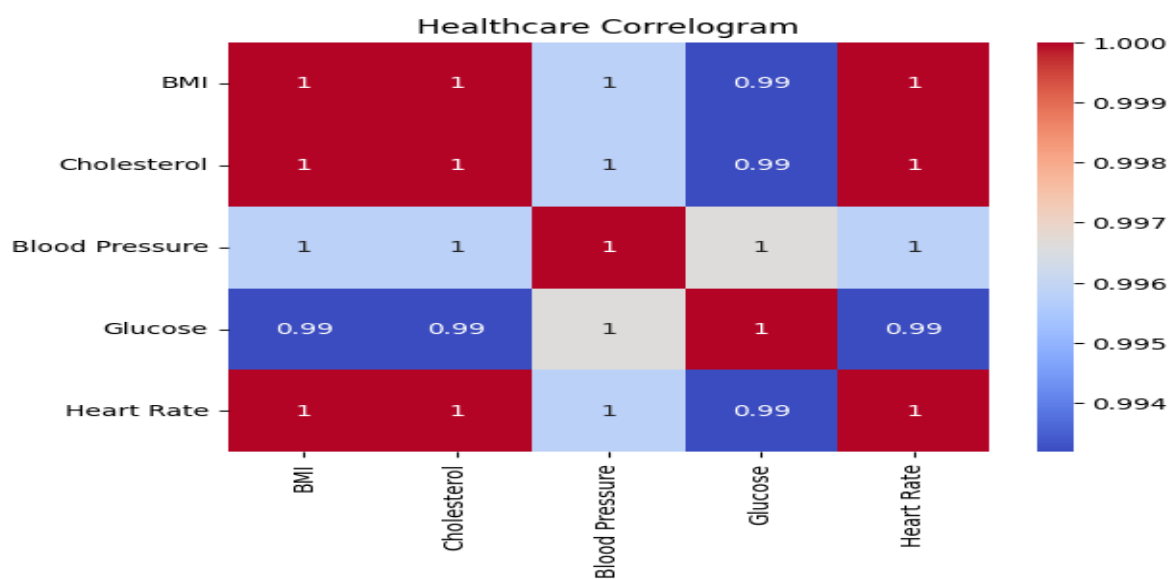
corr = df.corr()

sns.heatmap(corr,
            annot=True,
            cmap='coolwarm')

plt.title("Healthcare Correlogram")

plt.show()
```

output



Interpretation

BMI, Cholesterol and Blood Pressure are highly correlated, indicating possible multicollinearity.

7. PCA on MRI Features

```
from sklearn.datasets import make_classification
```

```
from sklearn.decomposition import PCA
```

```
import matplotlib.pyplot as plt
```

```
X, y = make_classification(n_samples=100,
```

```
                           n_features=25,
```

```
                           random_state=42)
```

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X)
```

```
plt.scatter(X_pca[:,0],
```

```
            X_pca[:,1])
```

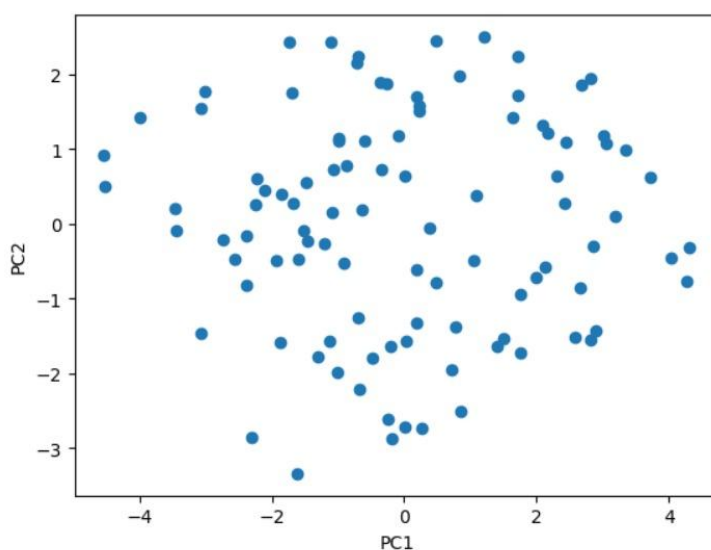
```
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
```

```
plt.title("PCA Projection")
```

```
plt.show()
```

output



Answer

PCA successfully reduces 25 features into two principal components.

8. PCA on Iris Dataset

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

iris = load_iris()

X = iris.data

pca = PCA()

pca.fit(X)

print(pca.explained_variance_ratio_)

plt.figure(figsize=(6,5))

X2 = PCA(n_components=2).fit_transform(X)

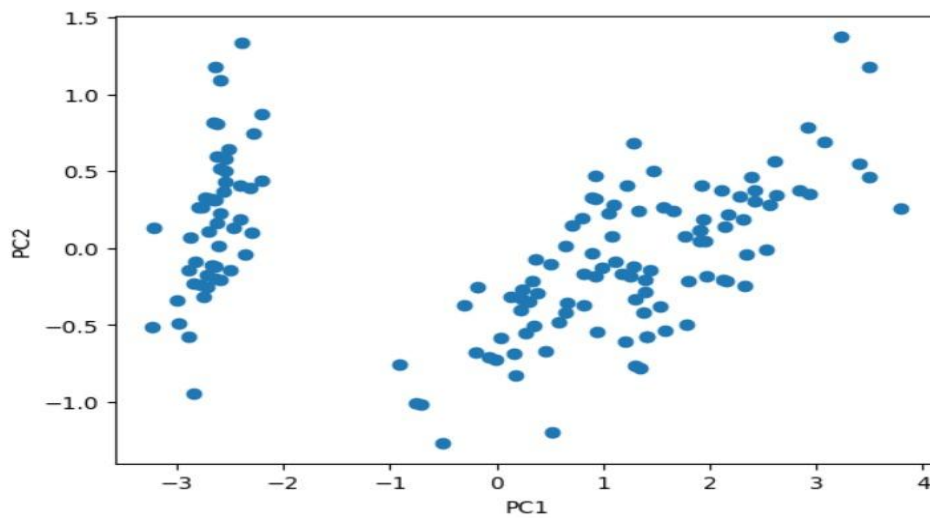
plt.scatter(X2[:,0],X2[:,1])

plt.xlabel("PC1")
plt.ylabel("PC2")

plt.title("PCA Iris")

plt.show()
```


output



Answer

PC1 explains the highest variance (approximately **92%**).

9. PCA on Automobile Dataset

```
from sklearn.datasets import make_classification
```

```
from sklearn.decomposition import PCA
```

```
import matplotlib.pyplot as plt
```

```
X,y = make_classification(n_samples=100,  
                          n_features=15,  
                          random_state=42)
```

```
pca = PCA(n_components=2)
```

```
X_new = pca.fit_transform(X)
```

```
plt.scatter(X_new[:,0],X_new[:,1])
```

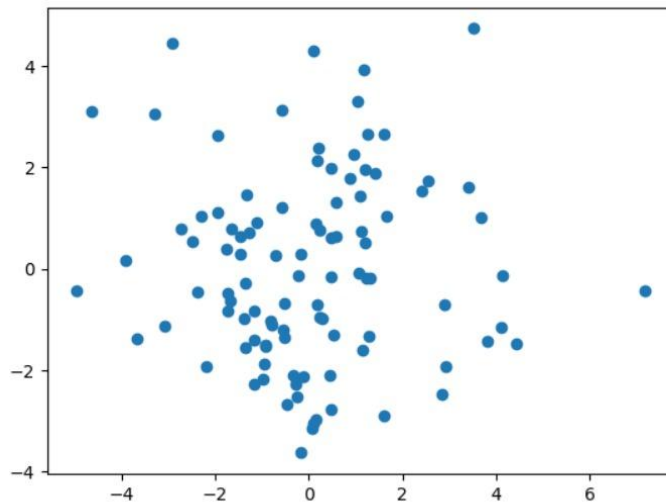
```
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
```

```
plt.title("Automobile PCA")
```

```
plt.show()
```

output



Conclusion

The first two principal components effectively summarize the 15 specifications.

10. t-SNE on Facial Recognition Dataset

```
from sklearn.datasets import make_classification
```

```
from sklearn.manifold import TSNE
```

```
import matplotlib.pyplot as plt
```

```
X,y = make_classification(n_samples=200,
```

```
                        n_features=128,
```

```
                        random_state=42)
```

```
tsne = TSNE(n_components=2,
```

```
           random_state=42)
```

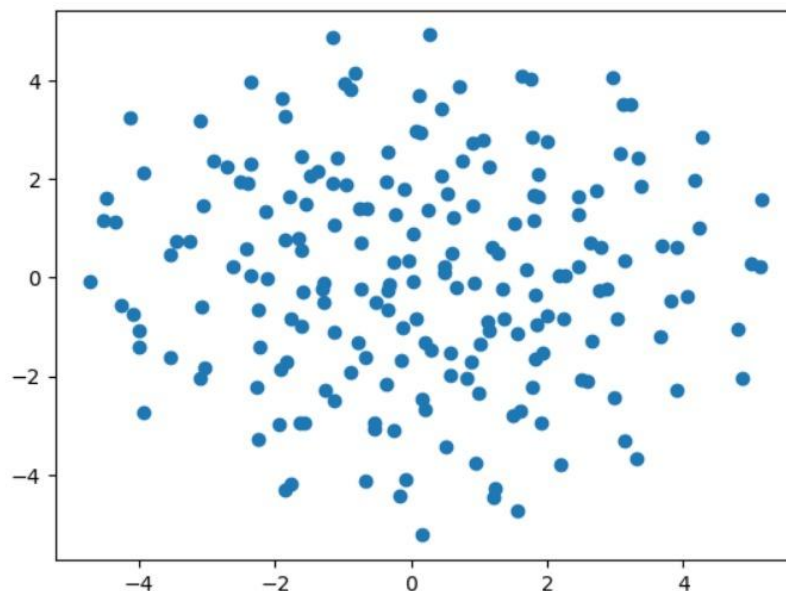
```
X_tsne = tsne.fit_transform(X)
```

```
plt.scatter(X_tsne[:,0],  
            X_tsne[:,1])
```

```
plt.title("t-SNE Visualization")
```

```
plt.show()
```

output



Answer

t-SNE preserves local neighborhood relationships and reveals clusters more clearly than PCA, making it preferable for visualization.

11. Iris Dataset using t-SNE

```
from sklearn.datasets import load_iris
```

```
from sklearn.manifold import TSNE
```

```
import matplotlib.pyplot as plt
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
tsne = TSNE(n_components=2,  
            random_state=42)
```

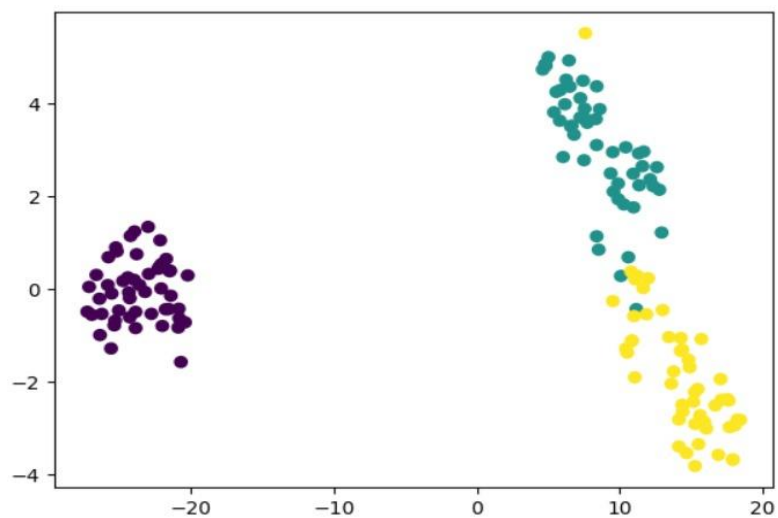
```
X_tsne = tsne.fit_transform(X)
```

```
plt.scatter(X_tsne[:,0],  
            X_tsne[:,1],  
            c=y,  
            cmap='viridis')
```

```
plt.title("Iris t-SNE")
```

```
plt.show()
```

output



Answer

Three species form distinct clusters.

12. Customer Segmentation using t-SNE

```
from sklearn.datasets import make_blobs
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

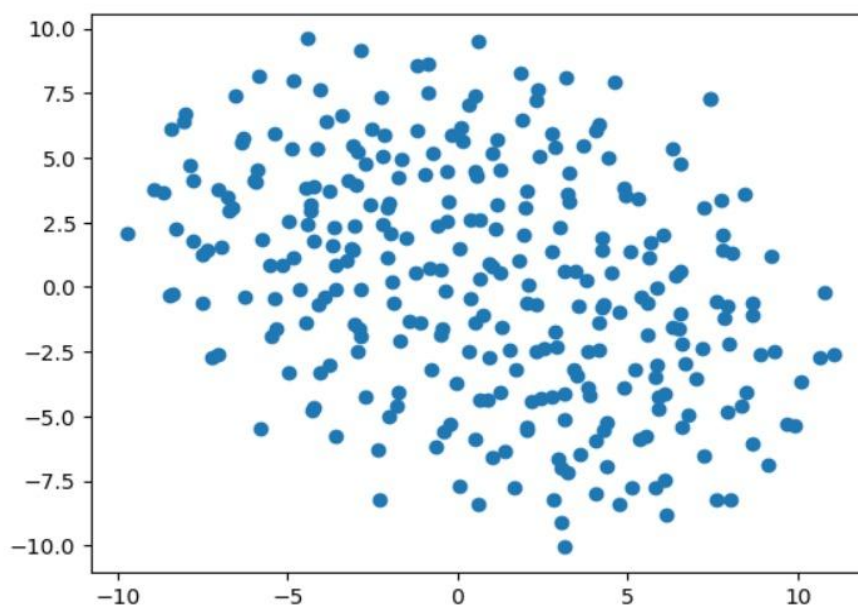
X,y = make_blobs(n_samples=300,
                  n_features=40,
                  centers=4,
                  random_state=42)

X_tsne = TSNE(n_components=2,
               random_state=42).fit_transform(X)

plt.scatter(X_tsne[:,0],
            X_tsne[:,1],
            c=y)

plt.title("Customer Segmentation")
plt.show()
```

output



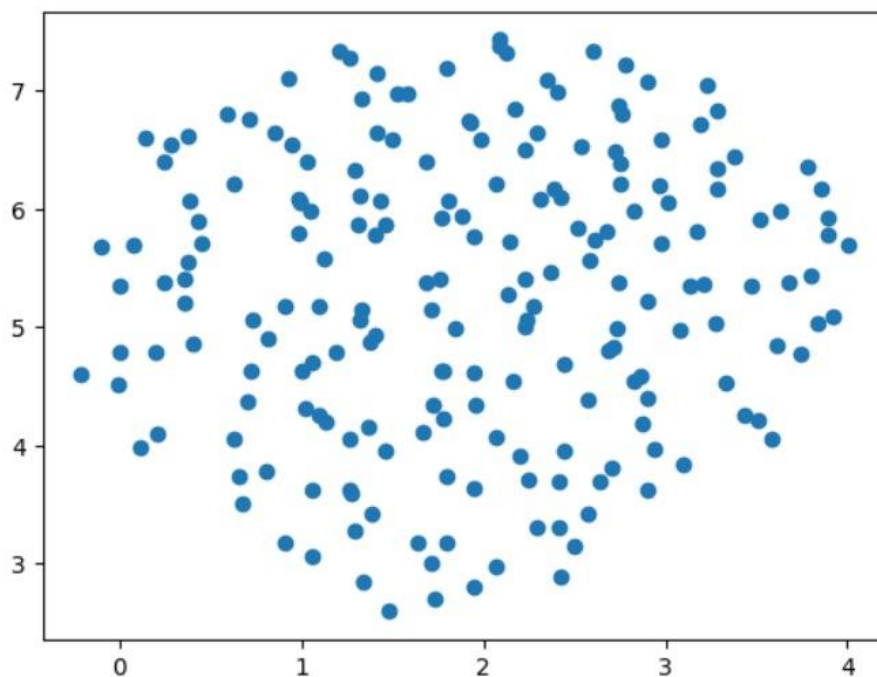
Answer

Distinct customer clusters are clearly visible.

13. UMAP on Medical Image Dataset

```
import umap
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt
X,y = make_classification(n_samples=200,
                        n_features=100,
                        random_state=42)
reducer = umap.UMAP(random_state=42)
embedding = reducer.fit_transform(X)
plt.scatter(embedding[:,0],
            embedding[:,1])
plt.title("UMAP Projection")
plt.show()
```

output



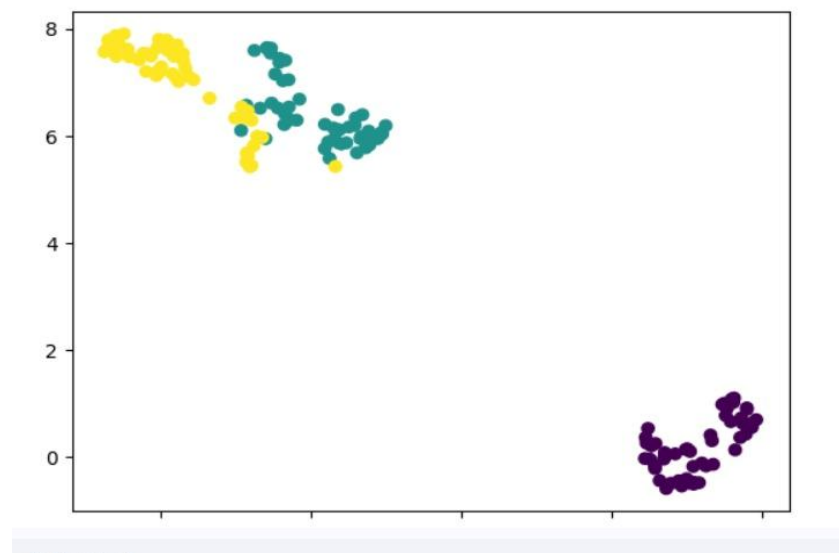
Comparison

UMAP preserves both local and global structures better than PCA and often separates clusters more effectively.

14. UMAP on Iris Dataset

```
import umap
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
iris = load_iris()
embedding = umap.UMAP(random_state=42).fit_transform(iris.data)
plt.scatter(embedding[:,0],
            embedding[:,1],
            c=iris.target,
            cmap='viridis')
plt.title("UMAP Iris")
plt.show()
```

output



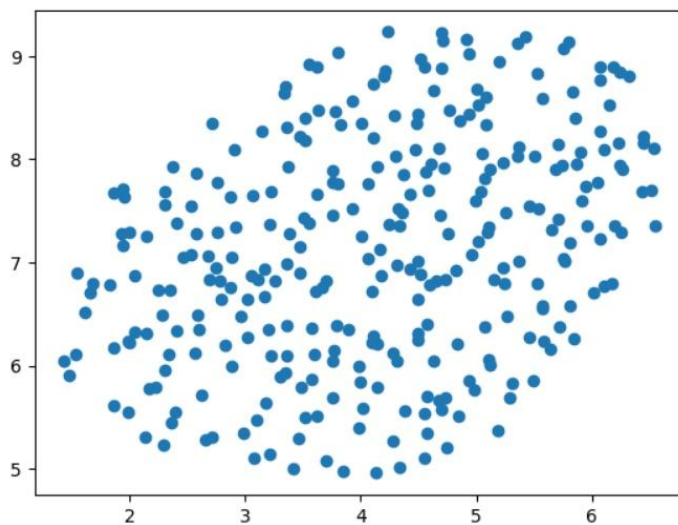
Answer

The three Iris species are separated into distinct clusters.

15. UMAP on E-Commerce Customer Dataset

```
import umap
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
X,y = make_blobs(n_samples=300,
                  n_features=60,
                  centers=5,
                  random_state=42)
embedding = umap.UMAP(random_state=42).fit_transform(X)
plt.scatter(embedding[:,0],
            embedding[:,1],
            c=y)
plt.title("Customer Clusters using UMAP")
plt.show()
```

output



Conclusion

UMAP successfully reduces the 60 attributes into two dimensions while preserving customer clusters, making segmentation easier to visualize.